

Architektury systemów komputerowych: Reference

Data: 2026-06-11
Zakres: 1 - 1

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	s = x ^ y c = x & y	Dodaje dwa bity. <code>s</code> to suma, <code>c</code> to bit przeniesienia (carry).
Pełny sumator (FA)	s = x ^ y ^ ci co=x&y ci&(x^y)	Jak półsumator, ale przyjmuje też przeniesienie wejściowe <code>ci</code> .
Sumator n-bitowy	$n \times \text{FA}$	Kaskada (ripple-carry): wyjście C_i trafia na wejście kolejnego FA. Ostatnie <code>co</code> to Carry Out całości.

Układy kombinacyjne

Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest tylko bit nr k .
Multiplekser	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjście przechodzi S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację (np. $A + B$, $A B$, $A\&B$); multipleksowanie wyników brankami AND/OR.

Układy sekwencyjne (pamięć)

Przerzutnik S-R	<code>S</code> - set, <code>R</code> - reset	Przechowuje 1 bit (Q). <code>S=1</code> ustawia $Q = 1$, <code>R=1</code> zeruje, <code>S=R=0</code> trzyma stan. Zbudowany z dwóch sprzężonych <code>NOR</code> -ów.
Sterowany poziomem	aktywny gdy <code>CLK</code> = 1	Wejścia <code>S</code> / <code>R</code> bramkowane AND-em z zegarem. Stan zmienia się tylko przy <code>CLK=1</code> .
Sterowany zboczem	zbocze 1 → 0	Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w momencie zmiany zegara.

2. Rejestry x86_64

<code>%rax</code> <code>%eax</code> <code>%ax</code> <code>%ah</code> <code>%al</code>	<code>%rbx</code> <code>%ebx</code> <code>%bx</code> <code>%bh</code> <code>%bl</code>
<code>%rcx</code> <code>%ecx</code> <code>%cx</code> <code>%ch</code> <code>%cl</code>	<code>%rdx</code> <code>%edx</code> <code>%dx</code> <code>%dh</code> <code>%dl</code>
<code>%rsi</code> <code>%esi</code> <code>%si</code> <code>%sil</code>	<code>%rdi</code> <code>%edi</code> <code>%di</code> <code>%dil</code>
<code>%rbp</code> <code>%ebp</code> <code>%bp</code> <code>%bpl</code>	<code>%rsp</code> <code>%esp</code> <code>%sp</code> <code>%spl</code>

<code>%r8</code>	<code>%r8d</code>	<code>%r8w</code> <code>%r8b</code>	<code>%r9</code>	<code>%r9d</code>	<code>%r9w</code> <code>%r9b</code>
------------------	-------------------	---	------------------	-------------------	---

Uwaga: Rejestry od `%r10` do `%r15` używają schematu:

`%r10` , `%r10d` , `%r10w` , `%r10b` .

Rejestry ogólnego przeznaczenia

<code>%rax</code>	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną z funkcji.
<code>%rbx</code>	Rejestr bazowy. Callee-saved.
<code>%rcx</code>	Licznik w pętlach i przesunięciach bitowych. 4. argument .
<code>%rdx</code>	Rejestr danych. Używany w mnożeniu/dzieleniu. 3. argument .
Indeksy i wskaźniki	
<code>%rdi</code>	Indeks docelowy (DI). 1. argument .
<code>%rsi</code>	Indeks źródłowy (SI). 2. argument .
<code>%rsp</code>	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki. Modfikowany przez <code>push</code> / <code>pop</code> / <code>call</code> / <code>ret</code> .
<code>%rbp</code>	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.

Nowe i specjalne rejestry

<code>%r8-9</code>	Kolejno: 5. i 6. argument . Caller-saved.
<code>%r10</code> oraz <code>%r11</code>	Rejestry tymczasowe. Caller-saved.
<code>%r12</code> do <code>%r15</code>	Ogólnego przeznaczenia. Wszystkie callee-saved .
<code>%rip</code>	Wskaźnik instrukcji (IP). Następna instrukcja. Modyfikowany przez skoki i wywołania.
<code>%rflags</code>	Rejestr flag statusu. Zmieniany przez <code>cmp</code> , <code>test</code> .

Rejestry wektorowe (zmiennoprzecinkowe)

<code>%xmm0</code> do <code>%xmm15</code>	128-bitowe rejestry. <code>%xmm0</code> to wartość zwracana (float/double). <code>%xmm0-%xmm7</code> to pierwsze 8 argumentów . Wszystkie są callee-saved .
<code>%ymm0</code> do <code>%ymm15</code>	256-bitowe rozszerzenie rejestrów XMM. Dzielą z nimi najmłodsze 128 bitów.

3. Flagi stanu (Rejestr %rflags)

ZF	Zero. Wynik to 0 (np. argumenty są równe).
SF	Sign. Wynik jest ujemny (MSB = 1).
CF	Carry. Przepelnienie dla liczb bez znaku (unsigned).
OF	Overflow. Przepelnienie dla liczb ze znakiem (signed).

4. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Immediate)	\$Imm	Imm
Rejestrowy (Register)	%Ra	%Ra
Bezpośredni (Direct)	Imm	M[Imm]
Pośredni (Indirect)	(%Rb)	M[%Rb]
Z przesunięciem (Indirect displacement)	D(%Rb)	M[%Rb + D]
Skalowany (Indirect scaled-index)	D(%Rb, %Ri, S)	M[%Rb+%Ri*S+D]
Skalowany (baseless)	(, %Ri, S)	M[%Ri * S]

Legenda: Imm / D to stała liczbowa, %Ra / %Rb to rejestr bazowy, %Ri to rejestr indeksowy, a S to skala (tylko wartości: 1, 2, 4 lub 8).

5. Konwencja wywoływań (System V AMD64 ABI)

Przekazywanie argumentów



Argumenty 7+ Na stosie od końca.
Wynik Zawsze w %rax .

Ochrona rejestrów (ABI)

Caller-saved (Można niszczyć)	Callee-saved (Musi przywrócić)
%rax , %rdi , %rsi , %rdx , %rcx , %r8 - %r11	%rbx , %rbp , %r12 - %r15

Ramka Stosu

Start (alokacja)	Koniec (sprzątanie)
<pre>push %rbp mov %rsp, %rbp sub \$32, %rsp</pre>	<pre>leave ret /* równowaznie: * mov %rbp, %rsp * pop %rbp * ret */</pre>

6. Struktury i wyrównanie

- Wyrównanie pola (K):** Zmienna K -bajtowa pod adresem p mod $K = 0$.
- Rozmiar całkowity:** Całkowity sizeof struktury musi być podzielny przez największe wyrównanie w strukturze ($K_{\max}$).

```
struct {
    char c; // 1 bajt (offset 0)
           // [3 bajty wewn. paddingu]
    int i; // 4 bajty (offset 4)
    short s; // 2 bajty (offset 8)
           // [2 bajty zewn. paddingu]
} // K_max = 4. Rozmiar: 1+3+4+2+2 = 12 bajtów.
```

7. Najważniejsze instrukcje (AT&T)

Opcode	Efekt	Opis
Przesyłanie danych		
mov S, D	$D \leftarrow S$	Kopiuje wartość z S do D.
push S	$\%rsp - = 8$ $M[\%rsp] \leftarrow S$	Odkłada na stos.
pop D	$D \leftarrow M[\%rsp]$ $\%rsp + = 8$	Zdejmuje ze stosu do D.
leave	$\%rsp \leftarrow \%rbp$ $\text{pop } \%rbp$	Sprząta ramkę stosu.

Adresowanie i arytmetyka		
lea S, D	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
add S, D	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
sub S, D	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
imul S, D	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
inc / dec D	$D \leftarrow D \pm 1$	Inkrementacja / dekrementacja.
neg D	$D \leftarrow -D$	Negacja arytmetyczna (jak w U2).

Dzielenie (zawsze w parze)		
cqto	$\%rdx:\%rax \leftarrow \text{sgn_ext}(\%rax)$	Uwaga: Rozszerza %rax do 128-bit przed idivq .
div S	$\%rax \leftarrow \text{wynik}$ $\%rdx \leftarrow \text{reszta}$	Dzielenie %rdx:%rax przez S. Wynik w %rax , reszta w %rdx .

Logiczne i bitowe		
and / or S, D	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
xor S, D	$D \leftarrow D \wedge S$	Często używane jako xor %rax, %rax do zerowania.
not D	$D \leftarrow \sim D$	Negacja bitowa (odwrócenie bitów).
sal / shl k, D	$D \ll = k$	Przesunięcie w lewo (mnożenie przez 2^k).
sar k, D	$D \gg = k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak.

Opcode	Efekt	Opis
<code>shr k, D</code>	$D \gg= k$	Logiczne w prawo. Dopełnia zerami.
Porównania i skoki		
<code>cmp S1, S2</code>	$S2 - S1$	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
<code>test S1, S2</code>	$S2 \& S1$	Ustawia flagi jak AND (np. test czy rejestr jest zerem).
<code>jX dest</code>	$\text{if}(X) \%rip \leftarrow \text{dest}$	Skok warunkowy (X = warunek).
<code>setX D</code>	$\text{if}(X) D \leftarrow 1$	Ustawia bajt (np. <code>%al</code>) na 0 lub 1 na podstawie flag.
<code>cmovX S, D</code>	$\text{if}(X) D \leftarrow S$	Warunkowe kopiowanie (optymalizacja zamiast skoku).
<code>call / ret</code>		Skok do funkcji / Powrót (obsługa stosu i <code>%rip</code>).

Operacje wektorowe		
<code>movaps / movups S, D</code>	$D \leftarrow S$	Kopiuje 128-bitów. <code>aps</code> (aligned) – wymaga wyrównania w pamięci, <code>ups</code> nie.
<code>movsd / movss S, D</code>	$D \leftarrow S$	Kopiuje pojedynczą wartość skalarną (<code>double</code> /(s) <code>float</code>).
<code>add/sub/mul/div pd/ps</code>	$D \leftarrow D \text{ op } S$	Arytmetyka wektorowa. Wykonuje operację na wielu parach równocześnie.
<code>add/sub/mul/div sd/ss</code>	$D \leftarrow D \text{ op } S$	Arytmetyka skalarna. Modyfikuje tylko najmlodsza wartość w rejestrze.
<code>vxorps S1, S2, D</code>	$D \leftarrow S2 \wedge S1$	Wektorowy XOR .
<code>v... (np. vmulss)</code>	$D \leftarrow S2 \text{ op } S1$	Przedrostek <code>v</code> wprowadza 3 argumenty (źródło 1, źródło 2, cel). Nie niszczy danych wejściowych.
<code>vfmadd231ss S1, S2, D</code>	$D \leftarrow S2 * S1 + D$	Fused Multiply-Add . Mnoży i dodaje. Cyfry <code>231</code> określają, że mnożymy dwa źródła, a wynik dodajemy do <code>D</code> .

Uwaga o sufiksach: Instrukcje mogą przyjmować sufiks określający rozmiar danych: `b` (byte, 8-bit), `w` (word, 16-bit), `l` (long, 32-bit), `q` (quadword, 64-bit). Np. `movq` kopiuje 64 bity, a `movl` kopiuje 32 bity (i zeruje górną połowę rejestru!).

8. Sufiksy warunkowe (dla <code>jX</code> , <code>setX</code> , <code>cmovX</code>)		
Sufiks	Synonim	Znaczenie / Warunek
Równość i znak (<code>ZF</code> , <code>SF</code>)		
<code>e</code>	<code>z</code>	Equal / Zero (równe / wynik to 0)
<code>ne</code>	<code>nz</code>	Not Equal / Not Zero (nierówne)
<code>s</code>		Sign (wynik ujemny, <code>SF=1</code>)
<code>ns</code>		Not Sign (wynik dodatni lub 0, <code>SF=0</code>)

Liczby ze znakiem (signed)		
<code>g</code>		Greater
<code>ge</code>		Greater or Equal
<code>l</code>		Less
<code>le</code>		Less or Equal

Liczby bez znaku (unsigned)		
<code>a</code>		Above (powyżej: <code>></code>)
<code>ae</code>	<code>nc</code>	Above or Equal / No Carry (większe równe: <code>>=</code>)
<code>b</code>	<code>c</code>	Below / Carry (poniżej: <code><</code>)
<code>be</code>		Below or Equal (mniejsze równe: <code><=</code>)

9. Optymalizacje i atrybuty		
Podstawowe techniki		
Strength red.	Zamiana drogich operacji na tańsze (np. <code>mul</code> $\rightarrow$ <code>lea</code> / <code>shl</code>).	
Loop unroll.	Rozwinięcie pętli (np. skok co 4. iterację). Mniej skoków = mniejszy narzut.	
Code motion	Wyrzucenie obliczeń niezmienników (np. stałych w pętli) przed pętlę.	
Common subexpr. elim.	Obliczenie wspólnego fragmentu tylko raz.	
Const folding	Obliczanie stałych wyrażeń (np. <code>2+2</code>) w czasie kompilacji.	
Branch predict	Sprzętowe przewidywanie skoków. By unikać kar, można użyć <code>cmovX</code> .	
Inlining	Wklejenie ciała funkcji w miejsce wywołania. Eliminuje narzut <code>call</code> / <code>ret</code> .	

Analiza pamięci i atrybuty GCC		
Aliasing	Problem nakładania się wskaźników. Keyword <code>restrict</code> , pomaga w optymalizacji.	
pure / const	Funkcja bez skutków ubocznych (<code>const</code>) dodatkowo nie czyta zmiennych globalnych).	

10. Bezpieczeństwo kodu i exploity		
Zagrożenia i błędy		
Buffer overflow	Zapis poza limit bufora. Nadpisuje sąsiadującą pamięć.	
Seg fault	Próba dostępu bez uprawnień (np. odczyt <code>NULL</code> , modyfikacja read-only).	

Stack smashing	Atak nadpisujący adres powrotu na stosie, by przejąć kontrolę po <code>ret</code> .
ROP (Gadżety)	Łączenie legalnych instrukcji kończących się <code>ret</code> w celu ominięcia zabezpieczeń.
Mechanizmy obronne	
Stack canaries	Losowa wartość ułożona przed adresem powrotu. Weryfikowana przed <code>ret</code> .
Nonexec code segments	Stos dostaje sprzętowy zakaz wykonywania kodu (tylko R/W).
Rand. stack offsets	Losowanie adresów stosu przy każdym starcie.